

Assignment 2: Compressing MobileNetV2

Nitin Sai Maradani [EE23B141]

CS6886: Systems Engineering for Deep Learning

September 8, 2026

1. Training Baseline

- a) Normalization via CIFAR10 mean = (0.4914, 0.4822, 0.4465), standard deviation = (0.2470, 0.2435, 0.2616)
- b) Data Augmentation:
 - `RandomCrop(32, padding=4, padding_mode="reflect")` : pads 4 pixels on each side, and randomly crops
 - Random Horizontal Flip
 - `RandomErasing(p=0.25, scale=(0.02, 0.2))`, with a probability of 25% erases 2-20% of the image - regularization
 - `mixup(images, targets, alpha=0.2)` : mixes up two images in the same batch
- c) MobileNetV2 is based on ImageNet (224×224), while CIFAR10 is 32×32 , so we decrease the strides of the first 2D Convolutions from 2 $\rightarrow$ 1
 - Optimizer: SGD, momentum 0.9, Nesterov, weight decay $5e-4$
 - LR schedule: linear warmup over the first 5 epochs, then cosine annealing down to 0 over the remaining epochs, base LR: 0.1
 - Regularization: weight decay (above) + label smoothing 0.1 + RandomErasing (in data augmentation) + dropout 0.2
 - Batch size: 128
 - Epochs: 150
 - Mixed precision (AMP) enabled
 - Seed fixed (42) for reproducibility
- d) **93.52%** is the test top-1 accuracy for the baseline model
- e) Failure modes:
 - Capacity Saturation of the model architecture: Any quantization below 4 bits, or pruning above 90% fails aggressively,
 - Hessian based pruning performs worse than magnitude-based, because the Δw^2 are extremely small, blurring the decision, ending up causing worse decisions than plain magnitude based,
 - Since this is a tiny model based on Depth-wise Seperable Convolutions, extreme pruning does not work well with this architecture (as they have very few parameters)

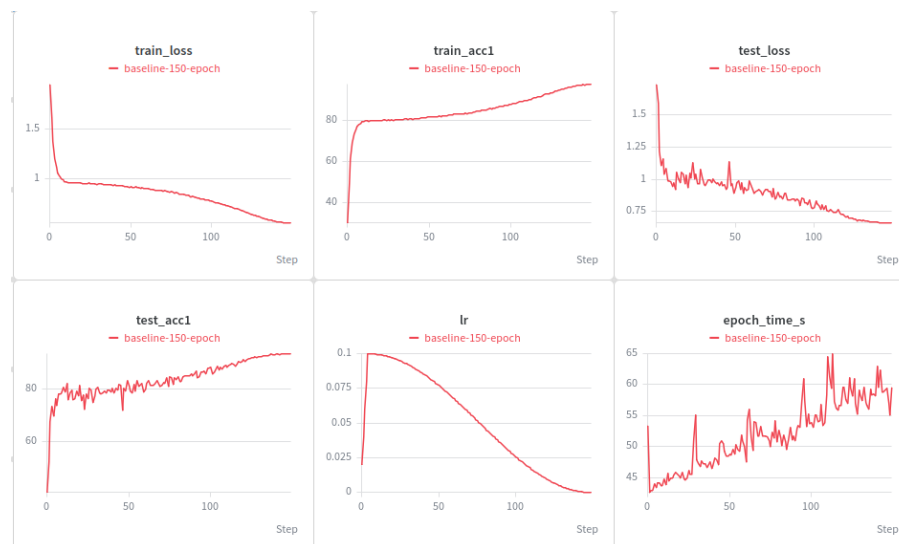


Figure 1: Baseline accuracy/loss/LR curves

2. Model Compression Implementation

I've implemented two compression techniques, quantization, and pruning.

1. **Quantization:** We need to quantize the weights and activations, quantizing weights is straight-forward, since weights are known before inference,
2. The same is not true for activations, hence we run a small batch of test-inference to calibrate the scale and bias for the activations, empirically found that a calibration size of 40 works well. (Increasing would definitely increase the performance, but is not worth the increase in compute)
3. Quantization is implemented via a "trick" of sorts in code: since PyTorch accepts only FP inputs, we "fake" quantize our weights and activations, i.e., we quantize and de-quantize, effectively injecting the quantization error into the values,
4. This is implemented by a "freeze" operation: `nn.Conv2d`, `nn.BatchNorm` and `nn.Linear` are replaced by their quantization counterparts in the model. When the quantization-counterparts are called: `quantize` → `de-quantize` → `freeze`, this freezes the weights with their quantized values in the model.
5. Activations are non-trivial to deal with, since they flow through the network, and keep changing
 - We insert a `ActFakeQuant` function after every `ReLU6`, i.e., activation,
 - But this leaves out one case, **Inverted Residuals**, it follows a simple logic that thin layers are not followed by a `ReLU6`, due to which there will be a `FP32` path that flows through, avoiding the insertion of quantization error,
 - So we just look for Inverted Residuals specifically and insert `textttActFakeQuant`
6. Pruning is implemented in a trivial manner: unstructured, magnitude based pruning. Prune off the bottom % weights.

7. Channel-wise quantization is applied to weights, compared to Tensor-wise quantization to activations. This is done because the distributions of weights can change significantly across channels.
8. The output of every module is quantized, since we cannot expect it to have the same scale/range as the input activations.

All layers are quantized, but Point-wise convolutions have majority of the parameters, and are largely pruned. Batch norm can be essentially be fused with the scale and shift. Batch Norm parameters are considered more sensitive, but did not bring any particular degradation to the model.

3. Compression Results

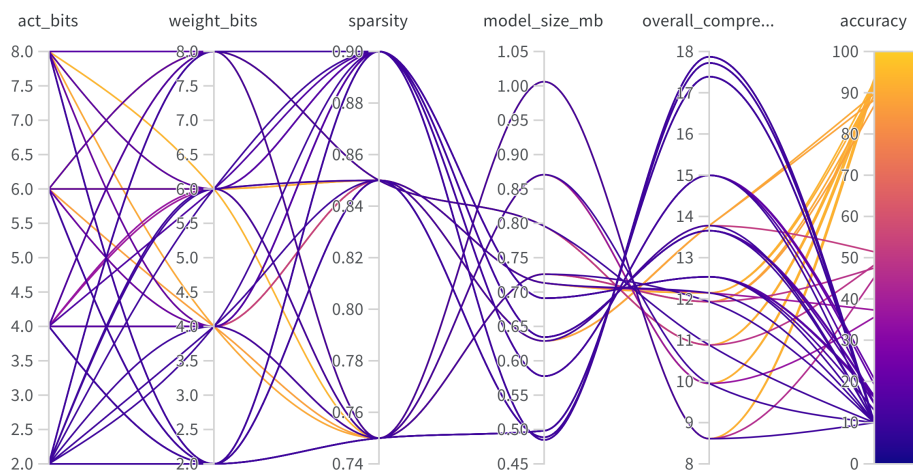


Figure 2: wandb parallel coordiantes plot

Documenting Storage & Metadata Overheads

Configuration: Sparsity = 0.85, Activation Bits = 6, Weight Bits = 4

Model Size_{FP32} = 8.66 MB (2.24M parameters + overheads)

Prunable Weights = 2.202M (excluding BatchNorm parameters)

$$\text{Sparse Weights Size} = \frac{2.202\text{M} \times (1 - 0.85) \times 6 \text{ bits}}{8 \times 1024^2} = 0.2362 \text{ MB}$$

$$\text{Sparsity Mask Overhead} = \frac{2.202\text{M} \times 1 \text{ bit}}{8 \times 1024^2} = 0.2625 \text{ MB}$$

$$\text{Number of Scales} = \sum C_{\text{out}} = 17,066$$

$$\text{Scale Overhead} = 0.0651 \text{ MB}$$

$$\text{Bias Size} = 17,066 \times 1\text{bit} = 0.0041\text{MB}$$

$$\text{Other FP32 Parameters} = 0.145 \text{ MB}$$

$$\text{Total Compression Ratio} = \frac{\text{Model Size}_{\text{FP32}}}{\text{Model Size}_{\text{Prune+Quant}}} = \frac{8.66\text{MB}}{0.713\text{MB}} = 12.15\times$$

Weight Bits	Act Bits	Sparsity	Compressed Accuracy	Total Compression
8	8	75%	93.36%	8.61×
8	8	85%	92.87%	10.88×
8	8	90%	15.91%	12.54×
8	6	75%	91.73%	8.61×
8	6	85%	91.35%	10.88×
8	6	90%	17.41%	12.54×
8	4	75%	53.49%	8.61×
8	4	85%	37.91%	10.88×
8	4	90%	11.07%	12.54×
6	8	75%	93.21%	9.95×
6	8	85%	92.88%	12.15×
6	8	90%	16.15%	13.65×
6	6	75%	91.59%	9.95×
6	6	85%	91.54%	12.15×
6	6	90%	17.05%	13.65×
6	4	75%	53.04%	9.95×
6	4	85%	49.73%	12.15×
6	4	90%	11.47%	13.65×
4	8	75%	89.41%	11.93×
4	8	85%	89.70%	13.78×
4	8	90%	18.77%	15.00×
4	6	75%	86.76%	11.93×
4	6	85%	87.22%	13.78×
4	6	90%	21.76%	15.00×
4	4	75%	51.08%	11.93×
4	4	85%	42.49%	13.78×
4	4	90%	13.71%	15.00×

Table 1: Experimental grid search across weight/activation bit-widths and target sparsities using the fixed pipeline parameter sweep.

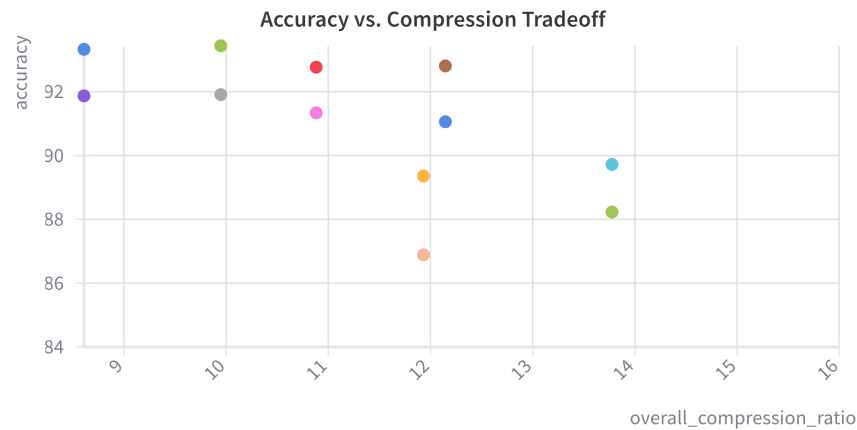


Figure 3: Scatterplot of Accuracy vs Compression (brown point is the chosen point)

4. Compression Analysis

From a pareto-optimal point of view (see fig. 3) weight bits = 6, activation bits = 8, sparsity = 85%, giving accuracy = 92.88% is the best. Since we are getting an extremely low accuracy drop ($\sim 0.64\%$) for $12.15\times$ compression.

- a) Compression Ratio of Weights = $16.25\times$
- b) Compression Ratio of Activations = $4\times$
- c) Accuracy of the Quantized Model = 92.88%
- d) Approximate size of the Quantized Model = 0.63 MB

Note: Although (activation, weights, sparsity, accuracy, compression) = (8, 6, 0.85, 92.88%, $12.15\times$) is a good pareto optimal point, 6-bit values are not "byte-aligned", hence are not hardware-friendly. For deployment on edge devices which are extremely compact, (activation, weights, sparsity, accuracy) = (8, 4, 0.85, 89.70%, $13.78\times$) is a better option, because besides offering a decently higher compression at the stake of accuracy, 4-bit weights are easier to implement, for both storage and MAC compute.

5. GitHub Repo

The complete implementation and experimental sweep are open-sourced on GitHub at - github.com/lazylettuce1/MobileNetV2-Compression.